CSC 436 Assignment

Semester DB Project: Phase III

Overview

In this phase, your group will revise and strengthen your database application based on the feedback received during the Database Expo. The goal of this phase is to improve the functionality, usability, database integration, and overall quality of your application before submitting your final polished project.

Your group should carefully evaluate the feedback you received, identify meaningful improvements, and implement revisions that strengthen both your database system and your application interface.

Part I: Review Feedback & Revision Plan

Carefully review the feedback your group received during the Database Expo, including peer comments, presentation observations, and any notes your group recorded during the event.

As a group, discuss the feedback and determine which revisions would most improve the quality of your database application. Your group should:

- Identify the strengths highlighted in your feedback.

  Several reviewers mentioned that the user interface was clean, intuitive, and easy to use. Other strengths included the chemical lookup feature, the dashboard, the inventory page, and the real-world usefulness of tracking chemicals in a lab environment. Reviewers also noted that the ER diagram and database design made sense, and that the application appeared to run smoothly during the demo.

- Identify any weaknesses, concerns, or suggested improvements.

  Some reviewers suggested making the frontend more user-friendly, adding low-stock or out-of-stock notices, improving password security through hashing, adding different user privileges for admins and students, adding filters or edit options, and making the ER diagram more organized.

- Determine which revisions are most important to complete before the final project submission.

Revision 1: Improve Password Security

We plan to update the login system so that user passwords are stored using password hashing instead of plaintext passwords. This revision directly responds to feedback that passwords were not hashed.

This change improves security and privacy by protecting user credentials in the database. Even if someone gains access to the database, the original passwords will not be directly visible. This also aligns the application with standard security practices.

Revision 2: Add Low Stock / Out-of-Stock Notices

We plan to add clearer status indicators for chemicals that are running low or have run out. This revision responds to feedback suggesting better visibility for inventory levels.

This change improves functionality by allowing users to quickly identify which chemicals need to be restocked. It also enhances the real-world usefulness of the application by supporting better inventory management.

- Document the specific changes your group plans to make.

  Our revision plan focuses on improving security and core inventory functionality while maintaining the existing strengths of the application. These changes strengthen the reliability, usability, and overall quality of the system without introducing unnecessary complexity before the final submission.

For each planned revision, **explain what will be changed**, **why the change is being made**, and **how the revision improves your database application**.

Focus on revisions that improve the overall functionality, usability, integration, reliability, security, and clarity of your project.

$2y$12$eojJJUGOgSh6aA6BYiXi3ubiZHv3p8C9DHzbUtZVCQd1D0YHXxlhW

Part II: Implementation Of Revisions

Implement the revisions and improvements identified in Part I. Your updates should be clearly reflected in your project materials, database, and application files.

Your group must revise and document changes to the areas of your project that were improved based on feedback. Depending on the feedback your group received, this may include:

- Updates to your database structure.

- Revisions to tables, relationships, integrity constraints, or validation checks.
- Improvements to queries or application/database interactions.
- Updates to the application interface to improve usability or clarity.
- Improvements to security or privacy protections.
- Refinements related to indexing, optimization or transaction reliability.

Part III: Documentation

Your submission should clearly document the revisions your group made. At minimum, your revised documentation should include:

A. Revision Explanations

Explain the revisions your group made based on feedback. For each major revision, describe what was changed, why it was changed, and how it improved the project.

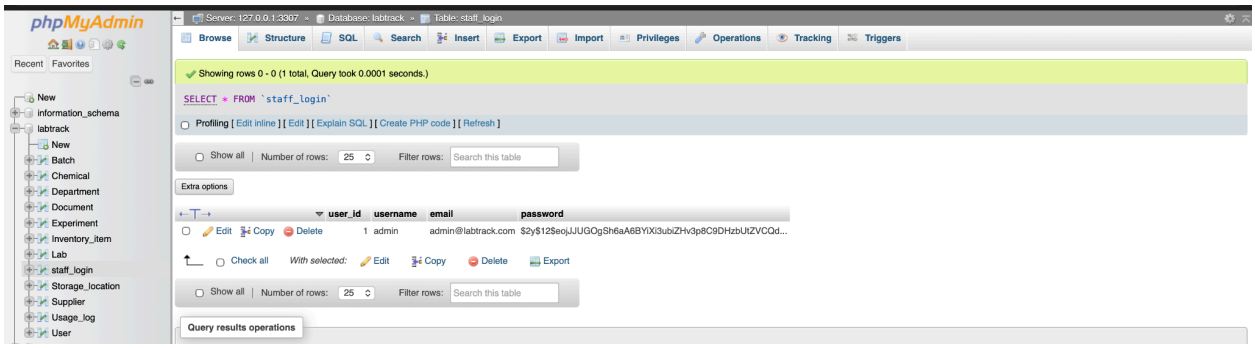
Revision 1: Password Security:
We updated the login system so that passwords are now stored as hashed values instead of plaintext. The original `admin123` password was replaced with a hashed password in the `staff_login` table, and the login code now uses `password_verify()` to check the entered password. This improves security because user passwords are no longer directly visible in the database.

Revision 2: Low Stock / Out-of-Stock Notices:
We updated the inventory system to automatically determine stock status based on quantity instead of relying on manually stored status values. Items with zero quantity are labeled "Out of Stock," items below a defined threshold are labeled "Low Stock," and all others are labeled "In Stock." This improves usability and reliability by ensuring that stock levels are always accurate and up to date, without requiring manual updates to the database.

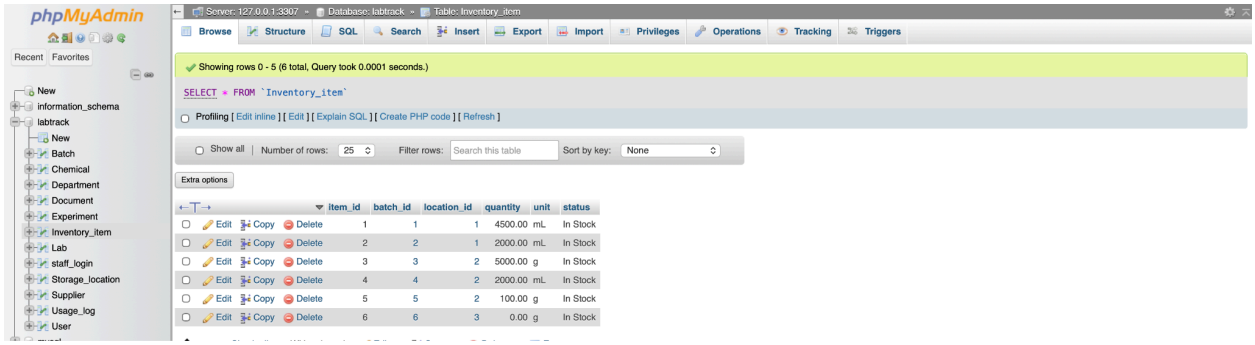
B. Updated Database Evidence

If you made revisions to your database structure or data, provide evidence of those changes. This may include screenshots of updated tables using the **Browse** tab, updated table structures, revised constraints, revised validation behavior, or other relevant database changes.

The `staff_login` table was updated so that passwords are stored as hashed values instead of plaintext. This can be seen in the database where the password field now contains a hashed string rather than the original password.



Additionally, the `Inventory_item` table remains unchanged structurally, but it is now used dynamically by the application to determine stock status based on quantity values.



C. Updated Query/Functionality Evidence

If you revised any SQL queries, application features, or interface/database interactions, provide evidence of those updates. This may include the actual SQL queries used, screenshots showing updated query results, screenshots of revised application functionality, or demonstrations of how the application and database now work more effectively together.

The authentication system was updated to use secure password verification. Instead of directly comparing plaintext passwords, the application now uses PHP's `password_verify()` function to validate user login attempts.

The inventory functionality was also improved by modifying the application logic to calculate stock status dynamically. Instead of relying on the `status` field stored in the database, the application determines whether an item is "In Stock," "Low Stock," or "Out of Stock" based on its quantity.

This change can be seen in the application interface, where items automatically display updated status labels when their quantities change. For example, items with low quantities are now clearly labeled as "Low Stock," improving visibility and usability.

Submission

When you're finished, complete the following steps to submit your work:

- ☑ ~~Export your document with responses as a **PDF file AND save** it **inside** your **documentation** folder. Refer to the following for documentation on how to do this:~~
  - Google Docs (*File → Download → PDF Document*)
  - Microsoft Word (*File → Save As / Export → PDF*)
  - Pages (*File → Export To → PDF*)
- ☑ ~~Export your updated database as an **SQL file** into the **db** folder. Follow the steps in the Exporting A Database section to export your database into a single SQL file.~~
- ☑ ~~Upload all your changes to GitHub.~~
  - ☑ ~~**If you're using GitHub Desktop (GUI)**, complete the Uploading Changes (GitHub Desktop) section to upload your changes from your local device to GitHub.~~
  - ☐ **If you're using Git (CLI)**, complete the Uploading Changes (GitHub CLI) section to upload your changes from your local device to GitHub.

ONE group member must paste the URL of your GitHub repository in the provided textbox in Brightspace. Click the blue *Submit* button to successfully submit your work for this assignment.

Grading Rubric

You can refer to the **Semester DB Project: Phase III grading rubric** given in Brightspace for this assignment to find details on how your submission will be graded.